OC Core Methods and Reproducibility Companion v1.3.3

Methods, validation, reproducibility, and replay companion

Alexander Yashin

Independent Researcher

ORCID 0009-0008-6166-0914

Version 1.3.3

Manuscript revision date: 4 May 2026

Concept DOI: 10.5281/zenodo.17899134

Dedicated to my dear wife Maria, without whom this work would have been impossible.

Acknowledgements

The author thanks the reviewers and critics whose questions, objections, and suggestions contributed to the development of the Ontology of Continua and to the refinement of this release. The acknowledged contributors are G. V. Apostolov, Eduard Fadeev, Gennady Alekseevich Nosov, Sergey Shpadyrev, and Stanislav Tsukrov. Their criticism helped sharpen the claim boundaries, improve the reader route, expose weak presentation choices, and force clearer separation between model claims, proof routes, numerical evidence, prior-art comparison, and publication form. Acknowledgement records review pressure and intellectual contribution to the development of the work; it does not imply authorship, endorsement, publication approval, responsibility for the theory, or agreement with any claim promoted in the manuscript.

Abstract

This methods companion explains how the formal, finite, numerical, and archival checks support the promoted claims. Many contemporary systems are too entangled to be understood by a single domain vocabulary: a technical platform carries physical infrastructure, software architecture, institutional incentives, security boundaries, user cognition, and economic feedback at the same time. The Ontology of Continua addresses that situation as a typed model-core project. It describes continuants, realizations, liveness, death, residue, boundaries, morphisms, operators, cycles, dimension, and K-level witnesses under explicit assumptions, so that a reader can ask how a system persists, changes, fails, and reappears without changing languages at every disciplinary border.

The project is intentionally ambitious in subject matter and intentionally bounded in claim discipline. Its publication task is not to replace physics, biology, cognition, systems engineering, enterprise architecture, or economics with a slogan. Its task is to offer a common structural grammar through which domain experts can compare state spaces, thresholds, flows, cycles, boundaries, and failure conditions without pretending that domain-specific science has become unnecessary.

Version 1.3.3 is a bounded external-review release. It is mature enough to be read as a coherent scientific manuscript and to be checked against formal and executable artifacts, but it remains a staged research program rather than a declaration of final all-domain completion. The release promotes model-core claims where their assumptions and evidence are visible; it leaves broader numerical closure, wider domain validation, and stronger comparative claims as future obligations.

The package contains a full monograph, a compact article route, a methods and reproducibility companion, a reviewer attack-and-response map, release navigation, public evidence anchors, finite-model outputs, Lean subset evidence, bounded target-blind replay QA, comparator and prior-art material, and appendices for proof, figures, tables, numeric rows, and source trace. The monograph is the canonical reading spine; the other documents are curated routes for first reading, editorial review, replay, and hostile criticism.

The evidence status is deliberately mixed and explicit. Some claims are supported by theorem routes, proof sheets, Lean declarations, and finite semantic witnesses. Some claims are supported by bounded numerical or target-blind replay rows that should be read as scoped QA evidence, not as complete empirical validation of a whole domain. Some claims are positioned against prior art or retained as research boundaries because the available evidence is not yet strong enough for a broader statement.

The principal limitation is therefore part of the manuscript's method: public wording must not

outrun the evidence. OC Core 1.3.3 asks the reader to evaluate whether the typed model, proof governance, finite semantics, numerical anchors, figures, tables, appendices, and reviewer-response routes form a coherent bounded model-core contribution. Future work must add evidence or mechanization before it widens the public claim surface.

The practical value of the manuscript is therefore a form of disciplined compression. If the model is useful, it should help readers translate complex systems into a shared ontology while preserving the meaningful distinctions that make each domain scientifically serious. That is the standard by which the release asks to be read.

Version 1.3.3 Release Delta

The OC Core release sequence is part of a continuing scientific program rather than a series of isolated archive events. As the model is tested against formal criticism, domain examples, reproducibility checks, and external review, the public manuscript is expected to become more precise, more explicit about its limits, and more useful to readers outside the author's immediate working context.

A new release is therefore warranted only when there is a meaningful scientific or editorial delta: a clarified definition, stronger proof route, better evidence boundary, improved reader pathway, repaired notation, new domain projection, or more honest comparator position. Minor process movement is not enough. The public release should tell readers what changed in the scientific object and why that change matters.

This policy is also a commitment to regularity without pretending that research can be made mechanical. The author intends to make future public updates systematic enough for readers, reviewers, and auditors to follow the development of the model, while preserving the difference between a public scientific result and private working material. No internal route, unpublished process detail, or control-plane record is promoted as reader-facing evidence merely because it helped produce the release.

Version 1.3.3 is the release in which the OC Core corpus is reorganized from a set of scattered source witnesses and process records into a reviewable scientific package. The visible delta is not merely a new archive number: the release strengthens the typed foundation, makes the claim boundary explicit, integrates the theorem route with public proof sheets, and separates reader-facing prose from machine evidence.

The model delta includes clearer treatment of carriers, realizations, liveness, death, residue, rebirth, morphisms, generalized boundaries, hybrid operators, cycle modes, historical and effective dimension, and K-level witnesses. These additions are presented as a bounded model-core grammar rather than as an unrestricted theory of every phenomenon.

The verification delta adds and consolidates a Lean-checked subset, structured proof sheets, finite semantic checks, finite witness interpretation, bounded target-blind replay QA, comparator and prior-art positioning, negative-control and falsifier language, and adversarial-review response material. Where a stronger claim is not yet earned, the release records the limitation instead of hiding it inside a source-control record.

The editorial delta is equally important. Figures, tables, formulas, numeric anchors, appendices, and evidence routes are kept in the publication corpus; machine coverage maps remain coverage and trace data only. The public documents are therefore built from the human manuscript hierarchy

and curated payload sources, while source bindings and machine evidence indexes stay in the review manifest.

For a returning reader, the practical point is this: 1.3.3 should be read as a clarification and consolidation release. It does not claim final closure. It improves the public surface through which the model can be criticized, taught, checked, and extended.

Reader Routes

Dear readers, systems theory is of interest to many communities, but rarely in exactly the same way. A formal reviewer, a philosopher of systems, an applied architect, an AI engineer, a security specialist, and an executive reader may all ask legitimate questions of the same manuscript. OC Core 1.3.3 is therefore designed as a deliberately generous scientific reading surface: it aims to disclose the Ontology of Continua as an applied model of system architecture while giving different readers a courteous route into the material.

Scientific reviewers and formal critics may wish to start with the abstract, release delta, model-foundation route, theorem/proof integration route, Lean subset, finite semantic witnesses, proof machinery, and falsifiability sections. This route is meant to make the work attackable in the best sense: every promoted claim should expose its assumptions, proof or executable evidence, counterexample boundary, and reopening condition.

Systems theorists, philosophers, and interested theoretical readers may prefer to start with the continuum problem, the historical and systems-theory motivation, the K-level primer, lifecycle and boundary chapters, prior-art comparison, and limitations. This path asks what OC inherits, what it reorganizes, where its residual delta begins, and where predecessors or parallel branches already carry part of the work.

Applied architects of complex systems–including engineers, enterprise architects, AI builders, security specialists, and applied researchers–may find it useful to start from practical examples, domain projections, figures, tables, and worked routes before returning to the formal core. The model chapters can then be read as a design grammar for carriers, realizations, boundaries, operators, update/flow separation, evidence classes, and claim gates.

CIOs, CEOs, enterprise architects, and strategy readers may sensibly read the release delta, practical utility chapter, model-comparison appendix, and final conclusion before investing in proof details. This route asks what the model is for, what it can support today, what remains too immature for operational commitment, and how evidence classes affect research or enterprise decisions.

Reproducibility auditors, evidence reviewers, and benchmark readers may go directly to the methods and evidence chapters, target-blind replay QA, numeric tables, machine-readable table reference, audit trail, source-trace appendix, and public evidence package manifest. This route asks whether formula, input snapshot, comparator, residual or uncertainty, negative control, falsifier, replay hash, and failure interpretation are present.

The companion is useful when commands, inputs, outputs, checksums, finite witnesses, target-blind replay, and failure interpretation are the main concern. In the full package, the conceptual model and formulas live in the monograph's scientific body; the proof route and Lean subset live in theorem, proof, and formalization sections; numeric evidence and target-blind replay QA live in the methods/evidence route; figures and tables live inline in the monograph; appendices carry

source trace, proof machinery, comparison rows, audit trail, and machine-readable table references; prior-art comparison and reviewer objections live in the article and attack-response map.

The author asks all readers to treat limitations as part of the claim, not as a footnote. A statement is promoted only where its assumptions, proof or evidence class, comparator boundary, and reopening condition are visible. Claims about complete scientific coverage, unrestricted all-domain numerical closure, or unrestricted superiority over contemporary science are not promoted by this release.

Table of Contents

1 Reading Map

This reading map is document-specific. The generated PDF table of contents gives page locations; the steps below state what the reader should do with each section.

1. Reproducibility method. Check the environment and dependency expectations.
2. Environment and inputs. Map each command to inputs, outputs, hashes, and claim impact.
3. Command replay. Run formal and finite checks before empirical replay.
4. Expected outputs and hashes. Interpret validation and simulation outputs locally.
5. Failure interpretation. Use failure interpretation to route defects to the right claim layer.
6. Claim affected by each check. Confirm checksum and archive identity after replay.

2 Reproducibility Structure and Audience

Purpose and role. This document is written for reviewers who want to replay the evidence locally. It exists to turn the release evidence into an executable methods protocol, so the opening pages identify the intended reader before they introduce formal claims.

Construction and order. The argument is organized as environment, archive layout, commands, inputs, outputs, expected hashes, and failure interpretation. The methods companion is procedural prose: each replay or check is tied to the claim class it can support or reopen.

The teaching obligation is that a reader should be able to decide which claim is threatened by each failed command. The current research support is the bounded OC Core 1.3.3 model-core stack: typed model, theorem and proof route, Lean subset, finite semantic witnesses, bounded replay rows, comparator positioning, phenomenon coverage, negative controls, falsifiers, and adversarial review. For the methods companion, the support stack is translated into replay inputs, expected outputs, and failure interpretation.

The document therefore states what the reader should learn from the evidence and where that evidence stops. It does not use release-readiness language as a substitute for scientific explanation, and it does not claim unsupported full-science completion.

3 Reproducibility Method

This companion is a methods guide, not the exhaustive finite-case register. The full machine-readable register is in the public evidence package; the prose here explains how a reviewer should replay the evidence and interpret failures.

3.1 Prerequisites

A reviewer needs two objects with different roles: the public release archive for curated evidence and citation, and a full repository checkout at tag v1.3.3 for executable command replay. The Zenodo public zip is intentionally curated; it is not a complete source checkout and should not be mistaken for the runner tree. Network access is not required for replaying already-pinned public evidence once the repository checkout and public archive have been downloaded.

Canonical source checkout route: clone the public repository, then check out the exact release tag.
The canonical repository is `https://github.com/alexanderyashin/ontology-of-continua-core-main`.
A reviewer may use `git clone https://github.com/alexanderyashin/ontology-of-continua-core-main.gi`

`cd ontology-of-continua-core-main`, and `git checkout v1.3.3`. The public archive DOI identifies the citable release object; the repository tag identifies the executable replay tree. The methods audit stops if those two identities disagree.

3.2 Environment Lock

The replay environment is treated as an explicit artifact boundary. The public archive records the repository manifest, checksums, Lean source, Lean build certificate, finite-model input facts, finite-model output attestation, target-blind table, validation report, simulation reports, and public payload suitability report. A reviewer should compare local command outputs with those named artifacts before interpreting scientific meaning.

The minimum local toolchain is Python 3, Pandoc with XeLaTeX for public PDF regeneration, Poppler `pdftotext` for PDF text audit, and Lean/Lake for the formal subset. Exact local package versions are not used as scientific evidence; they are replay conditions. If a tool version changes an output hash, the mismatch is a reproducibility finding until explained by a regenerated and audited artifact.

3.3 Public Archive to Repository Mapping

The public archive contains the human PDFs, curated evidence projections, metadata, checksums, and the release zip. The repository checkout contains executable directories such as formal/lean, proofs, validation, simulations, falsification, tools, and release_machine. A reviewer should use the archive to identify the published claim and checksum state, then use the checkout to run the commands against the same tag. If the archive and checkout disagree on version, DOI, manifest, or checksum, the replay stops at an archive-integrity finding before scientific interpretation begins.

3.4 Inputs

The replay inputs are the Lean source and certificate, theorem/proof registers, finite-model input facts, bounded numeric tables, validation reports, comparator registers, adversarial-review summaries, public checksums, and release metadata.

3.5 Commands

Run the replay from the repository root. The bounded public replay path is:

```
lake build OC133V12
python proofs/finite_model_checks/run_finite_model_checks.py
python validation/run_all.py --qa-only
python simulations/run_all.py --write-report
python simulations/adversarial/run_all.py
python falsification/counterexample_search/run_counterexample_search.py
python tools/oc133_public_release_payload.py --check
```

The expected outputs are the Lean certificate, finite-model report, replay QA report, simulation reports, adversarial simulation report, counterexample report, public payload suitability report, public zip checksum, and PDF text-audit files named in the manifest.

3.6 Replay Order

1. Build the Lean subset and inspect the Lean certificate.
2. Run the finite-model semantic checker and confirm that positive witnesses and negative controls separate as declared by the theorem boundary.
3. Run replay QA for bounded target-blind artifact-integrity and reconstruction examples.
4. Compare generated checksums with the public checksum manifest.
5. Treat any mismatch, parse failure, stale hash, unsupported claim, or publication-permission mismatch as a localized scientific or archive-integrity defect.

3.7 Expected Outputs

Expected outputs are a passing Lean certificate, a finite-model report with zero failures, replay QA rows with recorded residuals and falsifiers, simulation reports, a checksum manifest matching the archive, and no stale or private-path public-surface findings. If a command changes a generated artifact, the release is replayed and checksums are recomputed; if it changes nothing, delta-stable verification prevents downstream churn.

3.8 Failure Interpretation

A proof failure attacks a formal claim boundary. A finite-model mismatch attacks the executable semantic witness. A validation mismatch attacks a bounded replay row, not a unrestricted domain law. A checksum mismatch attacks package integrity. These failures are intentionally separated so that reviewers can identify the exact class of defect.

3.9 Artifact Map

Proof artifacts live under `proofs/` and `formal/lean/`; numeric replay artifacts live under `validation/` and `reports/`; reviewer artifacts live under `review/` and `reviews/`; publication artifacts live under `releases/oc_core_1_3_3/`. The public zip carries sanitized evidence projections for external replay.

4 Replay Matrix for Independent Review

This matrix is written in prose rather than raw table syntax so it remains readable in the PDF text extraction. Each item names the command, input files, output files, expected hash source, and claim consequence.

4.1 Lean formal subset

Command: lake build OC133V12. Inputs: formal/lean/OC133V12.lean and the Lake project files. Outputs: formal/lean/LEAN_BUILD_CERTIFICATE_1_3_3.json and the local Lake build result. Hash source: public checksum manifest and the Lean certificate hash recorded in the evidence package. Failure meaning: theorem claims citing the Lean subset are reopened until the declaration, proof boundary, or toolchain mismatch is repaired.

4.2 Finite semantic witnesses

Command: python proofs/finite_model_checks/run_finite_model_checks.py. Inputs: repository path proofs/finite_model_checks/OC133_FINITE_MODEL_INPUTS.json and related

finite-model runner code. Outputs: finite-model semantic report and the public finite-model output attestation. Hash source: checksums.txt, manifest.json, and the public evidence-package projection. Failure meaning: the theorem or semantic witness family named by the failing case is reopened; the whole model is not silently promoted.

4.3 Bounded target-blind replay QA

Command: `python validation/run_all.py --qa-only`. Inputs:

- `repository path validation/target_blind/OC133_TARGET_BLIND_PREDICTION_TABLE.json; public evidence path evidence/validation__target_blind__OC133_TARGET_BLIND_PREDICTION_TABI`
- `repository path validation/numeric_replay_qa/OC133_NUMERIC_REPLAY_QA_TABLE.json`
- public evidence projection `evidence/validation__target_blind__OC133_TARGET_BLIND_PREDICTION_TAB` Outputs:
- `repository path reports/OC_CORE_1_3_3_DOMAIN_VALIDATION_REPORT.json; public evidence path evidence/reports__OC_CORE_1_3_3_DOMAIN_VALIDATION_REPORT.json`
- the target-blind replay table
- the numeric replay QA table. Hash source: public checksum manifest, domain evidence-boundary report hash, and replay-table hash. Failure meaning: the affected replay row loses promotion; comparative or empirical wording depending on that row must be narrowed.

4.4 Simulation and adversarial simulation

Command: `python simulations/run_all.py --write-report`; `python simulations/adversarial/run_all.p` Inputs: simulations, adversarial simulation fixtures, and declared public model assumptions. Outputs: simulation reports and adversarial simulation summaries. Hash source: public package manifest and generated report checksums. Failure meaning: any claim that cites the affected simulation becomes provisional until the simulation report is repaired or the claim is demoted.

4.5 Counterexample search

Command: python falsification/counterexample_search/run_counterexample_search.py. Inputs: falsification/counterexample_search fixtures and declared theorem boundaries. Outputs: counterexample-search report with survivor or no-survivor status under the scoped. Hash source: public evidence package and checksums.txt. Failure meaning: a surviving counterexample reopens the exact claim boundary named by the search configuration.

4.6 Public payload audit

Command: `python tools/oc133_public_release_payload.py --check`. Inputs: public PDFs, public evidence package, metadata, checksums, DOI parameters, and known-error patterns. Outputs: PUBLIC_PAYLOAD_SUITABILITY_1.3.3_latest.json and PDF text-audit files. Hash source: public payload suitability report and generated archive checksum. Failure meaning: publication is stopped until the public-surface, archive, metadata, or claim-boundary defect is repaired.

A reviewer should read this matrix before running the commands. It states which scientific claim layer is threatened by each failure, so replay results do not collapse into an undifferentiated build status.

5 Claim-to-Command Interpretation

The methods companion must do more than name commands. Each replay command has a scientific role, an expected artifact, and a failure interpretation. This prevents a reviewer from treating the reproducibility path as an opaque build script.

5.1 Lean Build

`lake build OC133V12` checks the selected formal subset. A failure here does not merely indicate a tooling issue: it threatens every public theorem boundary that cites the Lean subset as support. The repair route is to inspect the Lean declaration, the theorem inventory binding, and the proof sheet assumption set before re-running the build.

5.2 Finite Semantic Checks

`python proofs/finite_model_checks/run_finite_model_checks.py` evaluates finite model facts independently from claimed verdict labels. The expected output is a report in which positive witnesses pass and mutation or tamper controls fail as designed. A mismatch threatens the semantic witness route for K0 resolution, lifecycle status, boundary classification, hybrid behavior, K-level transition, and minimality claims.

5.3 Validation QA

`python validation/run_all.py --qa-only` checks target-blind and numeric replay rows. A passing result means the row has the required formula, snapshot reference, split, prediction, observation, uncertainty, comparator, residual, negative control, falsifier, and replay hash. It does not mean that the whole scientific domain has been proved; it means the bounded public replay claim is internally auditable.

5.4 Simulation and Counterexample Search

The simulation and adversarial-simulation commands check whether the computational examples behave consistently with the stated model boundaries. The counterexample search is the negative side of the same method: it tries to find cases that break promoted assumptions. A serious counterexample does not get hidden in prose; it reopens the relevant claim boundary and routes the release back to Research and Review.

5.5 Public Archive Consistency Check

`python tools/oc133_public_release_payload.py --check` checks public PDFs, monograph integration, positive mission fulfillment, scientific/process state, editorial adversarial review, public surface language, known error regression, zip contents, and destination-ready metadata. It is expected to be delta-stable when inputs have not changed.

6 Evidence Inputs and Outputs

Every replay input has a provenance role. Lean sources and certificates support formal claims. Proof sheets support theorem assumptions and counterexample boundaries. Finite-model facts support semantic witnesses. Target-blind tables support bounded replay QA and artifact-integrity examples.

Domain evidence-boundary reports keep broad validation claims quarantined. Comparator registers support prior-art positioning. Reviewer summaries support adversarial closure. Checksums and manifests support archive integrity.

Every replay output must be interpreted at the same level as its input. A passing formal check supports formal consistency under declared assumptions; it is not an empirical result. A passing target-blind row supports a reconstruction claim; it is not a proof of unrestricted domain coverage. A passing public payload audit supports release presentation and packaging; it is not a substitute for theorem or data evidence.

7 Delta-Stable Verification

Verification should not dirty the release tree when no meaningful input changed. If a command rewrites a tracked artifact with identical semantic content, the release process treats that as a process-quality defect. If a command produces a real semantic delta, downstream packaging is triggered deliberately and the new checksum state is recorded. This prevents the release process from fighting itself while preserving strict reproducibility.

8 Reviewer Checklist

A methods reviewer can audit the release by asking seven questions. First, does each promoted claim have a replay or proof route? Second, does each command produce the declared artifact? Third, are negative controls and falsifiers present where the claim requires them? Fourth, are broad domain claims kept out of the promoted surface? Fifth, do checksums bind the public archive? Sixth, does a repeated verification pass leave the tree unchanged when there is no semantic delta? Seventh, does any failure route to a named claim boundary rather than disappearing into a generic build status?

Only if those questions are answerable from the public artifacts should the methods companion be considered publication-grade. That is why this PDF includes the interpretation protocol in prose rather than leaving readers with a list of commands and filenames.

9 Replay Visual Route

The methods companion uses visual route panels as operational diagrams. They are not decorative figures: the tuple panel tells the reader what object the replay concerns, the lifecycle panel tells the reader which status transitions are being tested, the K-level panel explains transition witnesses, the boundary panel explains classifier and falsifier checks, and the evidence panel shows how a claim moves from prose to proof, finite witness, numeric row, and reviewer reopening condition.

10 Artifact Interpretation Map

The methods reader should treat the public archive as a layered argument. The master monograph is the long-form scientific claim. The journal core is the article-length route. The methods companion is the replay route. The reviewer map is the adversarial route. The public zip is the evidence bundle. The manifest and checksum files bind the file set. A defect in one layer has a different meaning from

a defect in another layer, so the review protocol separates them rather than collapsing everything into a single pass/fail label.

If the master monograph fails, the scientific exposition itself is defective even if all machine files exist. If the journal core fails, the article-facing argument is not ready for editors even if the monograph is long. If the methods companion fails, the evidence is not reviewable even if the claims are plausible. If the reviewer map fails, hostile objections are not answered in public-facing form. If the public zip fails, the archive cannot be trusted as a reproducibility object. The release is acceptable only when all layers agree.

10.1 Traceability Requirements

Traceability is bidirectional. A claim in prose must point to an evidence route, and an evidence route must point back to the claim it supports. A theorem identifier must be meaningful in the theorem inventory, proof sheet, Lean subset where applicable, finite witness where applicable, and public explanation. A numeric row must be meaningful in the target-blind table, validation report, methods narrative, and claim boundary. A reviewer objection must name the claim under attack and cite the evidence that answers it.

The practical rule is simple: no orphan claims and no orphan evidence. An orphan claim is a public statement with no proof, data, simulation, comparator, or boundary. Orphan evidence is a file or row whose scientific role is not explained to the reader. Both are publication defects. The methods companion gives reviewers the route for detecting them before a public archive is updated.

10.2 Minimum Reproducibility Interpretation

Reproducibility is interpreted at four levels. Level one is file integrity: assets match checksums and the zip contains the expected curated files. Level two is command replay: the listed commands run and produce the expected reports. Level three is semantic replay: positive examples and negative controls separate according to the claim boundary. Level four is public interpretation: the reader can understand what the replay result means without reverse-engineering internal machinery.

A release can pass level one and still fail as science if it only archives files. It can pass level two and still fail as argument if the text does not connect outputs to claims. It can pass level three and still fail as publication if the public documents are unreadable. It can pass level four only when the archive, commands, semantics, and prose reinforce one another. That is the standard applied to OC Core 1.3.3.

10.3 What a Failed Replay Means

A failed replay is not automatically a refutation of the whole model. It is a localized signal. The reviewer should first identify the artifact class, then the claim boundary, then the dependency path, then the appropriate repair route. A Lean failure routes to formalization and proof assumptions. A finite semantic failure routes to witness construction and theorem boundary. A numeric replay failure routes to data snapshot, formula, uncertainty, comparator, and falsifier review. A public presentation failure routes to manuscript integration, editorial review, metadata, or packaging.

This localization matters because it keeps the release scientifically honest. The public claim surface is strong only where the evidence is strong. When the evidence is bounded, the public wording is bounded. When the evidence fails, the claim reopens. The methods companion therefore protects ambition by making ambition testable rather than by weakening it into vague language.

11 Worked Audit Walk-Through

A reviewer who wants a concrete path can start with T133-K0-RES. The reviewer reads the theorem statement in the monograph, checks that the claim boundary is bounded, opens the proof sheet for assumptions and counterexample boundary, confirms that the Lean subset or finite witness reference is not empty, then runs the finite semantic checker. If the finite report shows that the declared witness passes and that tampered or inert variants fail as expected, the reviewer has a localized reason to accept the bounded K0-support claim. If any part of the route is missing, the claim is not publication-ready.

The same pattern applies to a numeric row. The reviewer selects a lane, reads the formula and support scope, verifies that the dataset snapshot reference exists, checks the target-blind split, compares predicted value, observed value, uncertainty, residual, and comparator residual, then inspects the negative control and falsifier. The row supports only the bounded reconstruction claim named in the table. It does not become evidence for total domain closure unless a separate domain-validation artifact proves that stronger statement.

For prior art, the reviewer should not ask whether OC has no predecessors. That would be a weak and unscientific question. The correct question is whether the release identifies the relevant predecessor families, states overlap honestly, names residual delta precisely, and prevents that residual delta from inflating into a priority or unrestricted-comparison claim. The comparator rows and novelty register are therefore read as boundaries on external speech as much as evidence for novelty.

For editorial quality, the reviewer should open the PDF directly, not only the manifest. The document must have title page, dedication, version, DOI, abstract, reader contract, table of contents, explanatory paragraphs, transition language, visual anchors, literature synthesis, claim boundary, and a clear route from prose to evidence. If the first visible experience is metadata or if the text is a raw register dump, the methods replay can pass and the release can still fail as a scientific publication.

This walk-through is intentionally repetitive at the method level because it encodes the invariant for future releases: select a claim, locate its evidence, replay or inspect the evidence, check the negative boundary, compare against prior art, and read the public wording against that support. The invariant applies to formal, computational, empirical, editorial, and release-metadata layers alike.

12 Protocol Matrix

The replay protocol is organized as a matrix rather than a linear checklist. The rows are formal proof, finite semantic witness, target-blind replay QA, simulation, adversarial simulation, counterexample search, public-payload audit, and archive checksum verification. The columns are input, command, expected artifact, scientific interpretation, failure meaning, and repair owner. A row is reviewable only when all six columns can be read from the public artifacts without consulting private notes.

Protocol row	Input class	Expected artifact	Scientific interpretation	Failure route
Lean subset	typed formal source	build certificate	selected formal declarations type-check under declared assumptions	formalization and proof boundary
Finite semantics	raw finite model facts	output attestation	witnesses and tamper controls separate semantically	theorem witness and evaluator repair
Replay QA	pinned numeric rows	validation report	bounded reconstruction row is auditable	data, formula, uncertainty, comparator, or falsifier repair
Simulation	executable examples	simulation report	examples remain within declared model behavior	model assumption or example repair
Counterexample search	adversarial fixtures	counterexample report	promoted boundary resists known search patterns	claim reopening or proof/data repair
Public payload	manuscript and metadata sources	suitability report	public archive is readable and claim-bounded	editorial, packaging, or metadata repair

The matrix is deliberately conservative. It does not let a strong result in one row compensate for a missing row elsewhere. A theorem can be formally tidy and still need a finite witness if the public claim cites one. A numeric row can replay cleanly and still fail if the public prose describes it as whole-domain validation. A package can be checksummed and still fail if the first public reading experience is a metadata dump.

13 Domain-Lane Interpretation

The empirical lanes in this release are read as bounded domain-lane examples. Each lane has to name a source snapshot, reconstruction formula, prediction or reconstruction target, observed value, uncertainty or residual, comparator, negative control, falsifier, and replay hash. The lane supports the claim that the released artifact can be audited at that target boundary. It does not by itself prove that the entire domain is numerically solved.

A physics row therefore has a different force from a chemistry row, a biology row, a systems row, or a mathematics row. The methods companion keeps these forces separated. Physics and chemistry rows are closer to numerical reconstruction examples; biological and systems rows are more exposed to modeling assumptions; mathematics rows are formal or finite-model anchors rather than empirical measurements. The public claim boundary must reflect those differences.

The reader should also distinguish target-blind replay from prospective prediction. Target-blind

replay can prevent a file from being a circular restatement of its target, but it is still bounded by the chosen snapshot, split rule, formula, and comparator. Prospective prediction would require a future target and a pre-registered scoring rule. The release therefore treats prospective prediction as a future research extension unless an artifact explicitly provides it.

14 Independent Rebuild Expectation

A serious reviewer should be able to rebuild the public evidence route independently. Independence here does not mean rewriting the whole theory. It means that the reviewer can take the public archive, rebuild the formal subset, replay the finite semantic checks, inspect numeric rows, compare checksums, and confirm that the public PDFs describe the same claim boundaries as the evidence files. The release fails if the reviewer has to infer missing links from private process memory.

The release also fails if verification changes the object being verified without a declared semantic delta. Reproducibility is not only about re-running commands; it is also about preventing the release process from manufacturing new artifacts during inspection. When a real input changes, regeneration is correct. When no meaningful input changes, the correct result is a clean repeat check.

15 Reviewer Burden and Author Burden

The author burden is stricter for ambitious claims. A modest claim may need only a clear definition and a local example. A theorem claim needs assumptions, proof route, and boundary. A computational claim needs executable semantics and negative controls. An empirical claim needs data provenance, split, formula, comparator, residual, uncertainty, negative control, falsifier, and replay hash. A publication-readiness claim needs a human-readable manuscript set, editorial review, archive metadata, and post-release verification.

This is why the methods companion belongs in the public release. It teaches reviewers how to interpret the evidence without letting the evidence become a pile of files. The method is itself part of the scientific object: it states what kind of support the release has, what kind of support it does not yet have, and what would have to happen for the claim boundary to move.

16 Audit Decision Rules

The audit decision rules are intentionally explicit because otherwise reviewers and release operators can talk past one another. A formal failure blocks theorem promotion. A semantic witness failure blocks the theorem route that cites that witness. A numeric residual outside the declared tolerance blocks the replay row, not the whole model, unless the promoted claim depends on that row as unrestricted evidence. An absent comparator blocks any comparative wording. An absent negative control blocks empirical promotion. Defective front matter or an unreadable first page blocks publication even when the scientific files exist.

These rules prevent false escalation and false comfort. False escalation would treat a local replay mismatch as refutation of every OC claim. False comfort would treat a green checksum as proof that the manuscript is scientifically persuasive. The release needs neither habit. It needs exact routing: which claim is attacked, which evidence object is implicated, which standard is violated, and which capability must repair it.

16.1 Evidence Reading Examples

Example one is a finite semantic witness. The reader starts with the public theorem identifier, verifies that the proof sheet names assumptions, checks the finite-model case identifier, then confirms that the evaluator derives the result from raw model facts. If changing a label without changing facts changes the result, the evaluator is defective. If changing the required facts changes the result as expected, the witness is meaningful.

Example two is a bounded replay row. The reader starts with the lane and target, then reads the formula, pinned snapshot, split, predicted value, observed value, uncertainty, comparator, residual, negative control, falsifier, and replay hash. The row earns only the scope written in the claim boundary. If the public text says more than the row supports, the text is wrong even when the row itself is technically reproducible.

Example three is a prior-art comparison. The reader starts with a comparator family, checks the overlap statement, asks whether the claimed residual difference is specific, then verifies that the release does not infer absence, priority, or unrestricted comparative victory from a narrow source sample. The comparison is strong when it is exact and bounded. It is weak when it becomes a slogan.

16.2 Minimum Independent Scientific Review Packet

A reviewer who does not want to rebuild every artifact can still perform a minimum independent review. The minimum packet is the master monograph, journal core, methods companion, reviewer map, theorem registry, proof sheets, Lean source, finite-model input and output files, target-blind replay table, domain evidence-boundary report, comparator register, public checksum file, and Zenodo/GitHub metadata. Those objects must agree on version, DOI, claim boundary, author/instrument roles, and file identity.

Agreement is checked materially, not by trust. The version string must be the same. The DOI must identify the same record. The checksum must match the downloaded file. The claim wording must match the evidence class. The proof route must be named where theorem language appears. The empirical route must be named where replay language appears. The journal package language must not imply submission unless a separate submission action actually occurred.

This minimum packet is the bridge between a full rebuild and a first editorial screen. It lets a journal editor, archive curator, or hostile reader detect the most serious release defects without pretending that a quick inspection is the same as full scientific acceptance.

17 Visual Route - Replay Route Figures

The methods companion uses the same routes as audit diagrams: each rendered image states what has to be replayed or inspected.

Figure 1. The typed-continuum diagram connects the basic object vocabulary to the place where a public claim must be located.

Figure 2. The lifecycle diagram separates liveness, death, residue, rebirth, and identity preservation.

Figure 3. The K-level diagram separates witness-bearing transitions from inert observables that must be demoted.

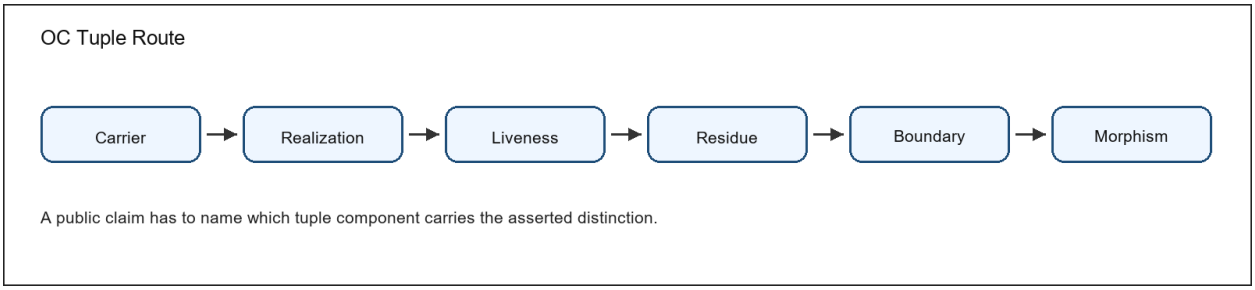

Figure 1: Conceptual diagram of typed OC continuum components

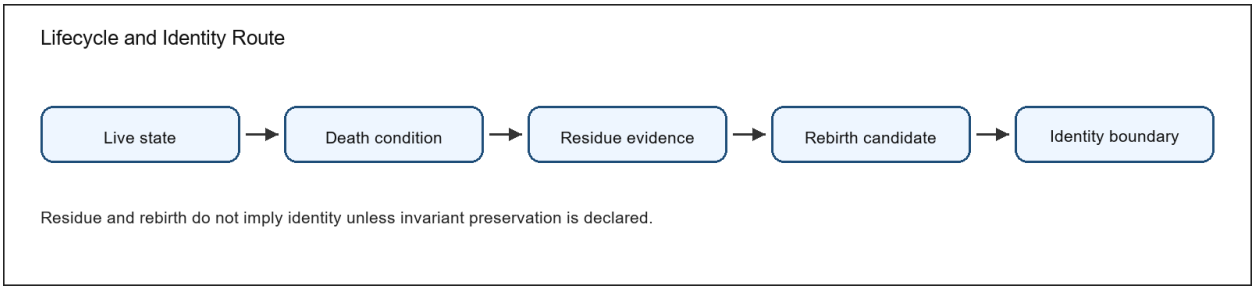

Figure 2: Conceptual diagram of lifecycle status and identity boundaries

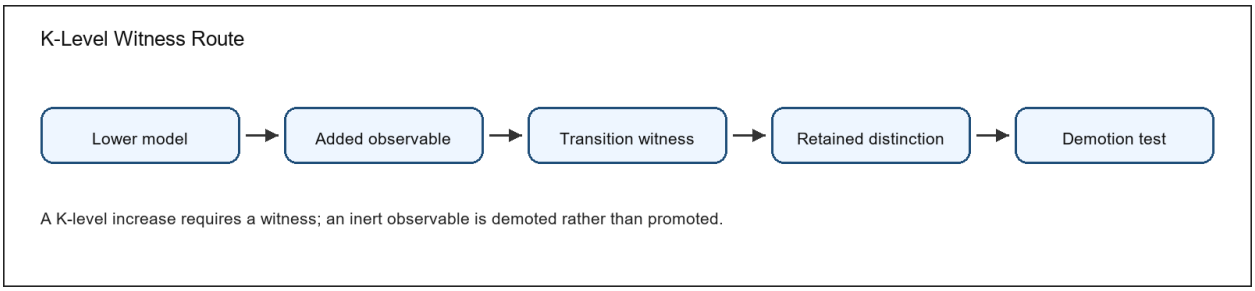

Figure 3: Conceptual diagram of K-level witness and demotion checks

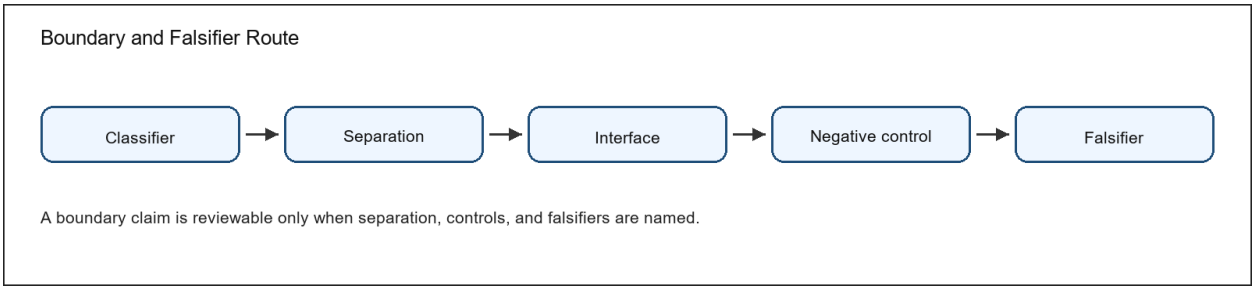

Figure 4: Conceptual diagram of boundary separation and falsifier checks

Figure 4. The boundary diagram links classifier, separation, interface, negative control, and falsifier.

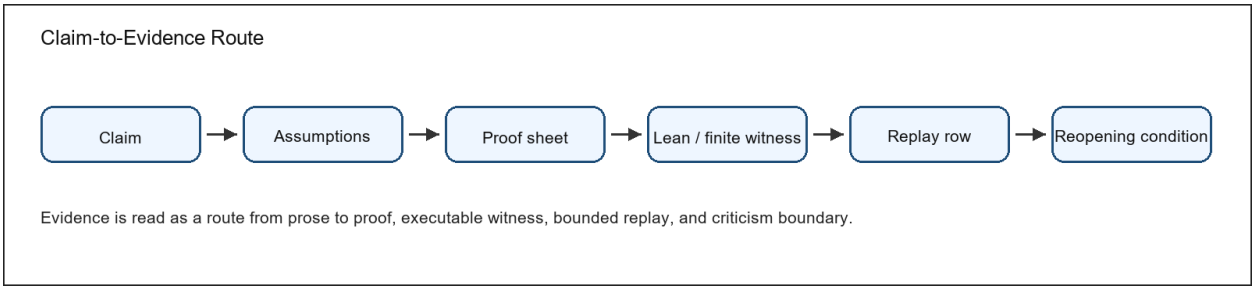

Figure 5: Conceptual diagram linking public claims to evidence and reopening conditions

Figure 5. The evidence diagram shows how a statement becomes reviewable instead of remaining a slogan.

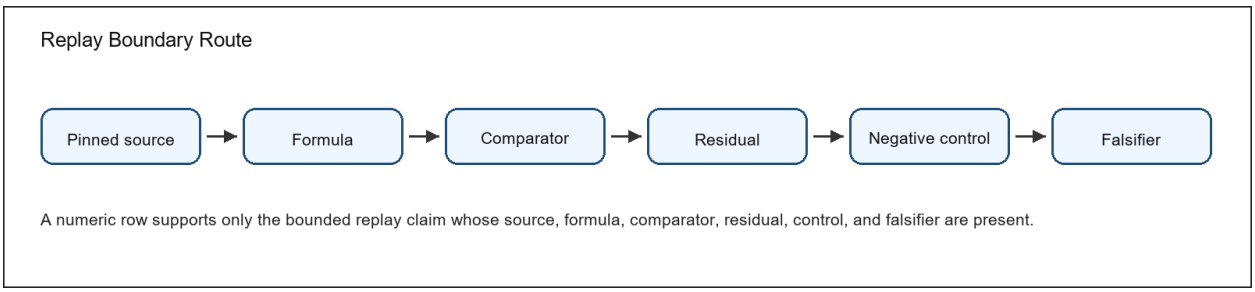

Figure 6: Conceptual diagram of bounded replay and empirical claim limits

Figure 6. The replay-boundary diagram shows why a numeric row is a bounded claim, not a whole-domain proof.

Tuple route. Carrier, realization, liveness, residue, boundary, and morphism prevent the tuple from being treated as a loose metaphor.

Lifecycle route. Live state, death condition, residue evidence, rebirth candidate, and identity boundary prevent residue from being confused with identity continuation.

K-level route. Lower model, added observable, retained witness, reduction test, and lawful demotion prevent hierarchy from being added without a witness.

Evidence route. Claim, assumptions, proof sheet, Lean or finite witness, replay row where applicable, and reopening condition prevent claims from being promoted without support.

Replay route. Source, formula, comparator, residual, negative control, and falsifier prevent numeric evidence from being over-read.

Route A: Tuple map Carrier -> realization -> liveness -> residue -> boundary -> morphism. The tuple is read left to right before theorem obligations are inspected.

Review use 1. The route points to the relevant definition, evidence artifact, and reopening condition.

Route B: Lifecycle route Live state -> death condition -> residue evidence -> rebirth candidate -> identity boundary. A failed invariant or residue mismatch blocks identity continuation.

Review use 2. The route points to the relevant definition, evidence artifact, and reopening condition.

Route C: K-level route Lower model + added observable -> transition witness -> retained distinction test -> demotion when the observable is inert.

Review use 3. The route points to the relevant definition, evidence artifact, and reopening condition.

Route D: Boundary route Classifier boundary -> observable separation -> negative control -> falsifier. Metric thresholds are special cases, not the whole boundary theory.

Review use 4. The route points to the relevant definition, evidence artifact, and reopening condition.

Route E: Evidence route Claim -> assumptions -> proof sheet -> Lean subset or finite witness -> numeric replay row when applicable -> reviewer reopening condition.

Review use 5. The route points to the relevant definition, evidence artifact, and reopening condition.

Route F: Replay boundary Pinned source -> formula -> comparator -> residual -> negative control -> falsifier. A numeric result carries only the claim whose replay boundary is complete.

Review use 6. The route points to the relevant definition, evidence artifact, and reopening condition.

18 Related Work and Comparator Boundary

The methods companion treats prior art as a reproducibility boundary: comparison claims require source rows, not general confidence. The release does not claim absence of predecessors, global priority, or unrestricted comparative claim over modern science. Its defensible public contribution is narrower: it integrates typed model-core claims, proof sheets, finite semantic witnesses, bounded numeric replay QA, comparator rows, and explicit reopening conditions into one auditable scientific release surface.

Required comparator family: Mereology and mereotopology. Boundary: OC does not claim to invent part-whole or boundary theory. It uses typed boundaries, residue relations, and classifier rules as the release-local way to keep boundary claims falsifiable.

Required comparator family: Process ontology and continuity. Boundary: OC does not settle every process-metaphysical debate. It states lifecycle, death, residue, rebirth, and identity-continuation boundaries under declared assumptions.

Required comparator family: Formal logic, category theory, type theory, and proof assistants. Boundary: OC uses these traditions as comparison and implementation context; a Lean declaration or finite witness supports only the exact bounded statement it encodes.

Comparator tradition: General System Theory. Source anchor: Ludwig von Bertalanffy, General System Theory. OC accepts the overlap: general systems framing and cross-domain system concepts. The bounded residual delta for this release is release-bound typed theorem register plus executable finite witnesses, numeric replay QA, falsifier registry, and authorization-bounded release-governed publication controls. The boundary is equally important: OC must not claim invention of general systems theory or organized-whole analysis.

Comparator tradition: Autopoiesis. Source anchor: Maturana and Varela, Autopoiesis and Cognition. OC accepts the overlap: autopoietic organization of living systems. The bounded residual delta for this release is typed distinction between liveness, death, residue, rebirth, and identity invariants. The boundary is equally important: OC must not claim invention of autopoiesis or self-producing organization.

Comparator tradition: Dynamical Systems. Source anchor: Encyclopedia of Mathematics, Dynamical system. OC accepts the overlap: mathematical dynamical-system state evolution. The bounded residual delta for this release explicitly blocks differentiating non-smooth proof/rewrite states unless smooth charts are declared. The boundary is equally important: OC must not claim invention of state spaces, flows, or attractor-style dynamics.

Comparator tradition: Category and Topos Formalisms. Source anchor: nLab, topos. OC accepts the overlap: category/topos formalisms and internal logic. The bounded residual delta for this release is the use of typed morphism discipline to police public scientific claims, without claiming invention of category theory. The boundary is equally important: OC must not claim invention of typed objects, morphisms, or topoi.

Comparator tradition: RAF Theory. Source anchor: Hordijk and Steel, Autocatalytic sets and boundaries. OC accepts the overlap: RAF formalization of autocatalytic sets and boundary discussion. The bounded residual delta for this release is the typed placement of RAF-like closure as one K-level route with explicit reduction and demotion checks, without replacing RAF theory. The boundary is equally important: OC must not claim invention of autocatalytic-set closure.

Comparator tradition: Complexity and Information Measures. Source anchor: Stanford Encyclopedia of Philosophy, Information. OC accepts the overlap: information concepts and measures. The bounded residual delta for this release is the release-local practice that separates historical activation from effective rank in the release theorem inventory. The boundary is equally important: OC must not claim invention of information or complexity measures. The full comparator matrix and source snapshots remain in the evidence package. If a future systematic search shows that a comparator already carries the same claim at the same strength, the OC public wording must be demoted or rewritten rather than defended by novelty rhetoric.

19 Observed Reproducibility Bill of Materials

This section records the build-host toolchain used to generate this public package. Long values are rendered as wrapped lines instead of table cells, because a public methods PDF must remain readable after PDF extraction.

Python.

```
Python 3.14.3
```

Pandoc.

```
pandoc 3.8.3
```

XeLaTeX.

```
MiKTeX-XeTeX 4.16 (MiKTeX 25.12)
```

Poppler pdftotext.

```
pdftotext version 24.04.0
```

Lean/Lake.

```
Lake version 5.0.0-src+7e01a1b (Lean version 4.28.0)
```

Git commit.

882a6ef

19.1 Public Output Paths

Master monograph. canonical long-form scientific text.

releases/oc_core_1_3_3/artifacts/OC_CORE_1_3_3_MASTER_MONOGRAPH_EN.pdf

Release guide. public landing and reading-order document.

releases/oc_core_1_3_3/artifacts/00_OC_CORE_1_3_3_RELEASE_GUIDE_EN.pdf

Journal core. compact article path.

releases/oc_core_1_3_3/artifacts/OC_CORE_1_3_3_JOURNAL_CORE_EN.pdf

Methods companion. replay and failure-interpretation protocol.

releases/oc_core_1_3_3/artifacts/OC_CORE_1_3_3_METHODS_AND_REPRODUCIBILITY_COMPANION_EN.pdf

Reviewer map. hostile-review route.

releases/oc_core_1_3_3/artifacts/OC_CORE_1_3_3_REVIEWER_ATTACK_AND_RESPONSE_MAP_EN.pdf

Public zip. archive and replay package.

releases/oc_core_1_3_3/artifacts/oc_core_1_3_3_public_release.zip

Expected hashes are not copied by hand into this paragraph; the authoritative hash surface is checksums.txt and the public zip manifest generated in the same build. A reviewer should compare those hashes to downloaded assets before interpreting scientific results.

20 Claim Boundary and Research Limits

OC Core 1.3.3 is a bounded external-review scientific release. It contains a typed model foundation, theorem/proof evidence, a Lean-checked subset, finite-model semantics, bounded numeric replay QA rows, comparator positioning, and adversarial-review material.

The release does not claim complete scientific coverage, unrestricted numeric closure, or unrestricted comparative victory over contemporary science. Those statements are outside the promoted 1.3.3 public claim surface.

Journal owner-review packets are included as preparation material only. They help editors and reviewers see how a later submission could be assembled, but they are not part of the scientific proof of the model core.

21 Data and Source Authority Notes

The methods companion treats empirical authorities as replay inputs, not as ornamental bibliography. Physics rows refer to official physical constants and pinned public-source snapshots. Chemistry rows refer to public chemistry property sources and compound/property snapshots. Biology rows refer to public biological datasets or expression snapshots. Systems rows refer to public indicator time series. Each row must keep source identity, access or snapshot date, formula, comparator, residual, negative control, falsifier, and replay hash together.

Formal source references, data-source records, and bibliographic records are normalized in the public evidence package and metadata files. This PDF explains how to read them during replay.

22 Environment Version Capture

A reviewer should record local versions before interpreting a mismatch: Python runtime, Lean/Lake version, Pandoc version, XeLaTeX distribution, Poppler version, operating system, repository commit, public zip checksum, and DOI record. These values are not scientific conclusions, but they decide whether a difference is a model defect, a replay-environment defect, or an archive-integrity defect.

Repository commit. This binds source files to public PDFs and manifests. A wrong commit means the replay is not testing the published object.

Public zip checksum. This binds the downloaded archive to the DOI record. A mismatch is an archive-integrity problem before it is a scientific problem.

Python runtime. This executes finite-model, validation, simulation, and packaging scripts. Version-sensitive output must be recorded as a replay-environment finding.

Lean/Lake. This checks the selected formal subset. A build failure reopens theorem boundaries that cite the formal subset.

Pandoc, XeLaTeX, and Poppler. These rebuild and audit public PDFs. Text or page-count drift is an editorial and reproducibility finding.

This version-capture rule prevents a common false inference. A local rebuild mismatch is not automatically a refutation of OC, but it is also not harmless. The reviewer first identifies which layer changed, then routes the finding to the claim, evidence, archive, or editorial surface that actually depends on that layer.

Keywords and Citation Route

Keywords: Ontology of Continua; continuum ontology; typed model core; systems theory; autopoiesis; dynamical systems; hybrid systems; formal methods; Lean formalization; finite semantic checks; target-blind replay QA; reproducible research; artifact evaluation; claim governance; falsifiability; evidence-bound scientific publishing.

Citation identity: Alexander Yashin, OC Core Methods and Reproducibility Companion v1.3.3, 4 May 2026. The Concept DOI is printed on the title page.

Open repository: https://github.com/alexanderyashin/ontology-of-continua-core-main. Repository files support reproducibility and source inspection; they do not replace the manuscript's claim boundaries.